

CAUCASUS UNIVERSITY · SCHOOL OF TECHNOLOGY

Python Programming II

Meeting 1 · Loops & Iterative Algorithms

for / while · guess-and-check · bisection search

Lecturer: **Prof. Maksim Iavich**

miavich@cu.edu.ge · Scientific Cyber Security Association



MEETING 1

Loops & Iterative Algorithms

Solving problems by repeating a controlled step

Task 1

Count occurrences of 'bob'

- Write a program that prints the number of times the string 'bob' occurs in s.
- For example, if s = 'azcbobobegghakl', then your program should print 2.

Libraries / focus: *Core Python — string slicing inside a loop*

Solution — counting 'bob'

```
s = 'azcbobobegghakl'
numBobs = 0
for i in range(1, len(s)-1):
    if s[i-1:i+2] == 'bob':
        numBobs += 1
print('Number of times bob occurs is:', numBobs)
```

Task 2

Longest substring in alphabetical order

- Write a program that prints the longest substring of s in which the letters occur in alphabetical order.
- For example, if s = 'azcbobobegghakl', then your program should print:
- **Longest substring in alphabetical order is: beggh**

Libraries / focus: *Core Python — comparing adjacent characters*

Solution — longest ordered substring

```
s = 'azcbobobegghakl'
test = s[0]
best = ''
for n in range(1, len(s)):    # have s[0] so compare to s[1]
    if len(test) > len(best):
        best = test
    if s[n] >= s[n-1]:
        test = test + s[n]    # add s[n] if greater or equal
    else:                     # otherwise start a new run
        test = s[n]
print("Longest substring in alphabetical order is:", best)
```

Guess-and-check with loops

- **Exhaustive enumeration: try every candidate answer until one works.**
- A while loop repeats while a condition is true; a for loop repeats over a range.
- We use these ideas below to compute squares and cube roots step by step.
- Key question each time: what is the stopping condition?

Task

Compute a square with a while loop

- Using a while loop and repeated addition, compute x·x for x = 3.
- Print it in the form 3*3 = 9.

SQUARE with while

Multiply x by itself using repeated addition.

```
x = 3
ans = 0
itersLeft = x
while (itersLeft != 0):
    ans = ans + x
    itersLeft = itersLeft - 1
print(str(x) + '*' + str(x) + ' = ' + str(ans))
```

Task

Cube root with a while loop

- Ask the user for an integer and find its cube root by counting up with a while loop.
- If the number is not a perfect cube, print that it is not one.

CUBE root (while)

```
x = int(input('Enter an integer: '))
ans = 0
while ans**3 < x:
    ans = ans + 1
if ans**3 != x:
    print(str(x) + ' is not a perfect cube')
else:
    print('Cube root of ' + str(x) + ' is ' + str(ans))
```

Task

Cube root for negatives too

- Extend the cube-root finder so it also works for negative integers using abs().
- Restore the sign of the answer at the end.

CUBE root, positive & negative (while)

```
x = int(input('Enter an integer: '))
ans = 0
while ans**3 < abs(x):
    ans = ans + 1
if ans**3 != abs(x):
    print(str(x) + ' is not a perfect cube')
else:
    if x < 0:
        ans = - ans
    print('Cube root of ' + str(x) + ' is ' + str(ans))
```

Task

Cube root with a for loop

- Find the cube root of 27 by trying each candidate guess in a range with a for loop.

CUBE root with a for loop

```
cube = 27
for guess in range(cube+1):
    if guess**3 == cube:
        print("Cube root of", cube, "is", guess)
```

Task

Approximate a cube root

- Approximate the cube root of 27 to within epsilon = 0.01 by stepping a tiny increment.
- Count how many guesses it takes.

Approximate solution — cube root

Step forward by a tiny increment until close enough.

```
cube = 27
epsilon = 0.01
guess = 0.0
increment = 0.0001
num_guesses = 0
while abs(guess**3 - cube) >= epsilon:
    guess += increment
    num_guesses += 1
print('num_guesses =', num_guesses)
print(str(guess) + ' is close to the cube root of ' + str(cube))
```

Bisection search convergence

- **Search space shrinks by half each guess:**
  - first guess: N/2
  - second guess: N/4
  - g-th guess: N / 2^g
- Guess converges on the order of $\log_2 N$ steps.
- Bisection works when the value of the function varies monotonically with the input.
- Code as shown only works for positive cubes > 1 — why?
- **Challenges → modify to work with negative cubes, and with x < 1.**

Task

Square root by bisection search

- Approximate the square root of 25 to within epsilon using bisection search.
- Halve the search interval [low, high] each step.

Example — square root by bisection

```
x = 25
epsilon = 0.01
numGuesses = 0
low = 1.0
high = x
ans = (high + low)/2.0
while abs(ans**2 - x) >= epsilon:
    print('low = ' + str(low) + ' high = ' + str(high) + ' ans = ' + str(ans))
    numGuesses += 1
    if ans**2 < x:
        low = ans
    else:
        high = ans
    ans = (high + low)/2.0
print('numGuesses = ' + str(numGuesses))
print(str(ans) + ' is close to square root of ' + str(x))
```

Task 3

★ NEW

Count failed logins in a log line

- You are given one line from an authentication log in the string log.
- Using a loop, count how many times the alert keyword 'FAILED' appears.
- A sudden spike of failures can signal a brute-force attack — print a warning if there are 3 or more.

Libraries / focus: *Core Python (loops, split) — first step toward log analysis*

Solution — scanning a log line

★ NEW

```
log = 'OK FAILED OK OK FAILED FAILED OK FAILED'
alert = 'FAILED'
count = 0
for word in log.split(' '):
    if word == alert:
        count += 1
print('FAILED attempts detected:', count)
if count >= 3:
    print('Warning: possible brute-force attempt!')
```

Task 4

★ NEW

Brute-force a 4-digit PIN

- **Ethics first: only run this against a device or account you own or are authorized to test.**
- Loop over every PIN from 0000 to 9999 and stop as soon as it matches the secret.
- Count how many guesses were needed — this shows why short PINs are weak.

Libraries / focus: *Core Python (for, range, zfill) — brute-force intuition*

Solution — exhaustive PIN search

★ NEW

```
secret = '0427'             # imagine this value is unknown
found = None
guesses = 0
for n in range(10000):
    pin = str(n).zfill(4)  # 0 -> '0000', 42 -> '0042'
    guesses += 1
    if pin == secret:
        found = pin
        break
print('Cracked PIN:', found)
print('Guesses needed:', guesses)
print('A 4-digit PIN has only 10,000 combinations - too few!')
```



END OF MEETING 1

Recap

Loops power everything from math to security automation